Wrocław
University
of Science
and Technology

Containers on AWS: ECR and ECS — Part 1

Cloud Programming (W04IST-SI0826G)

Wojciech Thomas, PhD

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026

This week

After this class, you should be able to:

- Explain the role of Amazon ECR and distinguish it from public registries
- Perform the full image lifecycle: build, tag, authenticate, and push to ECR
- Configure an ECR lifecycle policy and predict its effect on cost and risk
- Describe the ECS component hierarchy and map it to a two-layer application
- Construct a task definition for a two-container application
- Distinguish between ECS Fargate and EC2 launch types and justify the right choice

Why this matters

- A startup's Spring Boot backend runs as a container on a single EC2 virtual machine
- The team pushes its Docker image to Docker Hub — publicly accessible
- An attacker pulls the image and extracts **API keys embedded in the layers**
- The production database is wiped; customer data is gone
- The company spends three months rebuilding trust with regulators
- **One private registry and a scoped pull policy would have prevented this**

Wrocław University of Science and Technology

Agenda

Quick recall

Without looking at your notes:

- What is a Docker image? What is a container?
- What command starts multiple containers together locally?
- How does Docker Hub know whether you are allowed to push to a private repository?

- Image = read-only filesystem snapshot; container = running instance of an image
- `docker compose up` (or `docker-compose up`)
- Docker Hub uses **username/password or access token** authentication — not tied to any cloud IAM

Quick recall

Without looking at your notes:

- What is a Docker image? What is a container?
- What command starts multiple containers together locally?
- How does Docker Hub know whether you are allowed to push to a private repository?

- Image = read-only filesystem snapshot; container = running instance of an image
- `docker compose up` (or `docker-compose up`)
- Docker Hub uses **username/password or access token** authentication — not tied to any cloud IAM

Before we continue — predict

You push your container image to a **public** Docker Hub repository. An ECS service in your AWS account pulls and runs it.

What are the **two biggest risks** with this approach?

- **Exposure** — your proprietary code, config, and any embedded secrets are publicly readable by anyone who pulls the image
- **Reliability** — Docker Hub enforces pull rate limits for unauthenticated and free-tier users

Before we continue — predict

You push your container image to a **public** Docker Hub repository. An ECS service in your AWS account pulls and runs it.

What are the **two biggest risks** with this approach?

- **Exposure** — your proprietary code, config, and any embedded secrets are publicly readable by anyone who pulls the image
- **Reliability** — Docker Hub enforces pull rate limits for unauthenticated and free-tier users

Amazon ECR

- Elastic Container Registry — AWS managed private container registry
- Stores, versions, and serves OCI/Docker-compatible container images
- **Regional** — images are stored in the same AWS region as your ECS cluster
- Access controlled entirely by **IAM** — no separate registry credentials
- No rate limits for pulls within the same AWS account and region
- Integrated with ECS, CodePipeline, and CodeBuild out of the box

Amazon ECR

Fully managed private registry for Docker-compatible container images, integrated with AWS IAM for authentication and authorisation.

Wrocław University of Science and Technology

ECR vs. Docker Hub

	Amazon ECR	Docker Hub
Visibility	Private by default	Public by default
Authentication	AWS IAM (token via CLI)	Username / password or PAT
Pull rate limits	None (same account/region)	Yes — 100–200 pulls/6 h
AWS integration	Native — ECS, CodeBuild, IAM	Manual credential injection
Image scanning	Built-in (on push or on demand)	Paid tier only
Cost	~$0.10/GB/month storage + data transfer	Free tier; paid for private repos

Wrocław University of Science and Technology

Agenda

The image lifecycle

From local development to a running ECS task — four steps:

1. **Build** — `docker build -t backend:v1.0.0 .`
2. **Tag** — re-tag with the ECR repository URI
3. **Authenticate** — get a temporary ECR login token via the AWS CLI
4. **Push** — `docker push <ecr-uri>/backend:v1.0.0`

ECR stores the image

ECS pulls it at task (container) launch.

Authenticating to ECR

```
aws ecr get-login-password --region eu-west-1 | docker login \
  --username AWS \
  --password-stdin 123456789012.dkr.ecr.eu-west-1.amazonaws.com
```

- `get-login-password` calls IAM and returns a **12-hour** token
- The username is always `AWS`
- Token is piped into `docker login` — never stored in a file
- Required IAM permission: `ecr:GetAuthorizationToken`

Tagging strategy

- `:latest` — always points to the most recent push; **mutable**; risky in production
- **Semantic version** — `v1.2.3`, `v2.0.0-rc1` — clear, human-readable
- **Git SHA** — `7a3b4c9` — unambiguous, traceable to a commit

- **Immutable tags**
  - ECR repository setting
  - a pushed tag cannot be overwritten

Never deploy :latest to production!

You cannot tell which code is actually running.
Use Git SHA or version tags instead.

Wrocław University of Science and Technology

Agenda

Before we continue — predict

Your team pushes 10 images per day to an ECR repository. No lifecycle policy is configured.

- How many images are in the repository after 30 working days?
- What happens to the storage bill?

- 300 images — every push creates a new entry, nothing is removed
- ECR charges ~$0.10 per GB per month
- An image from three months ago looks identical to a current one in the list

Before we continue — predict

Your team pushes 10 images per day to an ECR repository. No lifecycle policy is configured.

- How many images are in the repository after 30 working days?
- What happens to the storage bill?

- **300 images** — every push creates a new entry, nothing is removed
- ECR charges ~$0.10 per GB per month
- An image from three months ago looks identical to a current one in the list

ECR lifecycle policies

- Rules that automatically expire images based on age or count
- Applied per repository; evaluated daily by AWS
- Two expiry types:
  - **Count-based** — keep only the last N images matching a rule
  - **Age-based** — expire images older than N days
- Match criteria: tag status (`tagged`, `untagged`, `any`), tag prefix
- Running ECS tasks are **not** protected — a lifecycle rule can expire an image that a task is still using; plan tag exclusions carefully

Wrocław University of Science and Technology

Agenda

The ECS component hierarchy

Four nested primitives — from the outside in:

- Cluster — logical boundary grouping services and compute resources
- **Service** — maintains a *desired count* of running tasks; restarts failed tasks
- **Task definition** — versioned blueprint: images, CPU, memory, ports, roles
- **Task** — one running instance of a task definition (the live containers)

ECS hierarchy

Cluster → Service → Task *(running instance of a Task Definition)*

Mapping ECS to the two-layer app

Our application has two services in one ECS cluster:

Component	ECS Service	Subnet	Container
Frontend	`frontend-service`	Public	Vite / Node.js
Backend	`backend-service`	Private	Spring Boot

- Each service has its own **task definition** and its own **IAM roles**
- Services communicate via ALB path-based routing (`/api/*` → backend)
- Direct container-to-container IP communication is possible but fragile — use the ALB (see the next lecture)

Admin challenge

Your colleague proposes: "Let's put the Vite frontend and the Spring Boot backend in a single task definition with two containers. That way they start together and share the same network namespace."
*What are the **two operational problems** with this approach?*

1. Scaling is coupled
2. Deployment is coupled

Admin challenge

Your colleague proposes: "Let's put the Vite frontend and the Spring Boot backend in a single task definition with two containers. That way they start together and share the same network namespace."
*What are the **two operational problems** with this approach?*

1. Scaling is coupled
2. Deployment is coupled

Wrocław University of Science and Technology

Agenda

Task definitions: the blueprint

- Analogous to a `docker-compose.yml` service block — but for AWS
- **Immutable** — every change creates a new **revision** (`:1`, `:2`, `:3` …)
- ECS services are pinned to a specific revision; rollback = update to a previous revision
- Key top-level fields:

Field	Purpose
Launch type / requires compat.	Fargate or EC2
Task CPU / memory	Total budget shared by all containers
Task execution role	IAM role used by the ECS agent
Task role	IAM role used by your application code

Inside a container definition

Each container in a task definition specifies:

```
Image URI:
    123456789012.dkr.ecr.eu-west-1.amazonaws.com/backend:v1.0.0
CPU:          256 (.25 vCPU — soft reservation)
Memory:       512 MB (hard limit — OOM kill if exceeded)
Port:         containerPort 8080
Environment:  SPRING_PROFILES_ACTIVE = production
Secrets:      DB_PASSWORD → arn:aws:secretsmanager:...:db-password
Log driver:   awslogs → /ecs/backend (CloudWatch log group)
```

Container dependency ordering

- Use `dependsOn` when container F must wait for container B

```
{
  "name": "frontend",
  "dependsOn": [{
    "containerName": "backend",
    "condition": "HEALTHY"
  }]
}
```

- Available conditions
  - **`START`** — wait until container A has started (process launched)
  - **`HEALTHY`** — wait until container A's health check is passing
  - **`COMPLETE`** — wait until container A exits successfully (init containers only)

Only relevant for multi-container task definitions. For separate services, the ALB health check enforces readiness.

Wrocław University of Science and Technology

Agenda

Fargate vs. EC2 launch types

	AWS Fargate	EC2 launch type
Infrastructure	Managed by AWS — no servers to patch	You manage EC2 instances in the cluster
Billing	Per task: vCPU + memory × seconds	Per instance: running even when idle
Startup time	~30–60 seconds (cold start)	~5–10 seconds (image already pulled)
Workload fit	Variable, unpredictable, bursty	Steady, high-throughput, cost-optimised
Spot support	Fargate Spot — up to 70 % discount	EC2 Spot instances
Operational burden	Low — no AMI updates, no bin-packing	High — instance sizing, capacity planning

Wrocław University of Science and Technology

Discuss with your neighbour (2 min)

You are advising two teams:

1. **Team A** runs a video transcoding pipeline — bursts of 200 tasks when uploads arrive, then zero tasks for hours
2. **Team B** runs a payment processing API — 50 tasks running 24/7, traffic predictable within ±10 %

- Which launch type would you recommend for each team?
- What is the key factor driving your recommendation?

Wrocław University of Science and Technology

Agenda

Wrocław University of Science and Technology

Closing

Before you go

On a piece of paper (or in your notes), write:

1. One thing from today you are **confident** about
2. One thing you are **still unsure** about — bring it to the lab

Self-check questions:

- Can you explain the four ECS primitives and how they nest?
- Can you describe the difference between a task definition revision and a running task?
- Can you justify why the backend service belongs in a private subnet?
- Can you explain why `:latest` is a risky production tag?

Wrocław University of Science and Technology

References

[1] Amazon Web Services, “Amazon elastic container service developer guide,” 2026. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ (accessed May 13, 2026).

[2] Amazon Web Services, “Amazon elastic container registry user guide,” 2026. https://docs.aws.amazon.com/AmazonECR/latest/userguide/ (accessed May 13, 2026).

[3] Amazon Web Services, “AWS fargate user guide for amazon ECS,” 2026. https://docs.aws.amazon.com/AmazonECS/latest/userguide/what-is-fargate.html (accessed May 13, 2026).

Wrocław University of Science and Technology

Thank you for your attention

- See you next week